



Sequences

Software Development COS7009–B

Sequence Types

1. Strings

- *Immutable*
- Conceptually very much like a tuple

2. List

- *Mutable* ordered sequence of items of mixed types

3. Tuple

- A simple *immutable* ordered sequence of items
- Items can be of mixed types, including collection types

4. Range()

- *immutable* sequence

Sequence Operations

- concatenate / +
- repeat / *
- indexing (the [] operator)
- slicing ([:])
- membership (the in operator)
- len (the length operator)

Contents - Strings

Strings

String Index

String Slicing

String properties: Iterable, Immutable, Operations

String methods

Sequence of Characters

- We've talked about strings being a sequence of characters.
- A string is indicated between ' ' or " "
- The exact sequence of characters is maintained

And then there is “““ ”””

- Triple quotes preserve both the vertical and horizontal formatting of the string
- Allows you to type tables, paragraphs, whatever and preserve the formatting

```
"""this is  
a test  
today"""
```

Non-printing Characters

If inserted directly, are preceded by a backslash (the \ character)

- new line `' \n '`
- tab `' \t '`

The Index (1 of 2)

- Because the elements of a string are a sequence, we can associate each element with an **index**, a location in the sequence:
 - positive values count up from the left, beginning with index 0
 - negative values count down from the right, starting with -1

The Index (2 of 2)

characters	H	e	l	l	o		W	o	r	l	d
index	0	1	2	3	4	5	6	7	8	9	10
									...	-2	-1



Accessing an Element

A particular element of the string is accessed by the index of the element surrounded by square brackets

```
hello_str = 'Hello World'  
print(hello_str[1]) => ??  
print(hello_str[-1]) => ??  
print(hello_str[11]) => ??
```

Contents - String

Strings

String Index

String Slicing

String properties: Iterable, Immutable, Operations

String methods

Regular Expression

Slicing, the Rules

- Slicing is the ability to select a subsequence of the overall sequence
- Uses the syntax `[start : finish]`, where:
 - `start` is the index of where we start the subsequence
 - `finish` is the index of **one after** where we end the subsequence
- If either `start` or `finish` are not provided, it defaults to the beginning of the sequence for `start` and the end of the sequence for `finish`

Indexing and Slicing (1 of 4)

```
helloString[6:10]
```

characters	H	e	l	l	o		W	o	r	l	d
index	0	1	2	3	4	5	6	7	8	9	10

↑ first

↑ last



Indexing and Slicing (2 of 4)

```
helloString[6:]
```

characters	H	e	l	l	o		W	o	r	l	d
index	0	1	2	3	4	5	6	7	8	9	10

↑ first

↑ last

```
helloString[:5]
```

characters	H	e	l	l	o		W	o	r	l	d
index	0	1	2	3	4	5	6	7	8	9	10

↑ first

↑ last



Indexing and Slicing (3 of 4)

```
helloString[-1]
```

Characters	H	e	l	l	o		W	o	r	l	d
Index	0	1	2	3	4	5	6	7	8	9	10
	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

↑
Last



Indexing and Slicing (4 of 4)

```
helloString[3:-2]
```

Characters	H	e	l	l	o		W	o	r	l	d
Index	0	1	2	3	4	5	6	7	8	9	10

↑ First ↑ Last



Extended Slicing (1 of 2)

- Also takes three arguments:
 - `[start:finish:countBy]`
- Defaults are:
 - `start` is beginning, `finish` is end, `countBy` is 1

```
my_str = 'hello world'
```

```
my_str[0:11:2] ⇒ 'hlowrd'
```

- Every other letter

Extended Slicing (2 of 2)

```
helloString[::2]
```

Characters	H	e	l	l	o		W	o	r	l	d
Index	0	1	2	3	4	5	6	7	8	9	10



The diagram illustrates the slicing operation `helloString[::2]`. It shows a sequence of characters and their corresponding indices. The characters are 'H', 'e', 'l', 'l', 'o', an empty space, 'W', 'o', 'r', 'l', and 'd'. The indices are 0 through 10. The characters at even indices (0, 2, 4, 6, 8, 10) are highlighted in light blue. Below the index row, five curved arrows indicate the step-by-step selection of characters at indices 0, 2, 4, 6, and 8, demonstrating the slicing operation with a step of 2.



Contents - String

Strings

String Index

String Slicing

String properties: Iterable, Immutable, Operations

String methods

Regular Expression

Strings are Immutable

- strings are immutable, that is you cannot change one once you make it:

- `a_str = 'spam'`
 - `a_str[1] = 'l' → ERROR`

- However, you can use it to make another string (copy it, slice it, etc.)

- `new_str = a_str[:1] + 'l' + a_str[2:]`
 - `a_str → 'spam'`
 - `new_str → 'slam'`

Sequences are Iterable

The for loop iterates through each element of a sequence in order. For a string, this means character by character:

```
for char in 'hello world':  
    print(char)
```

Membership Operations

- Can check to see if a substring exists in the string, the `in` operator. Returns True or False

```
my_str = 'aabbccdd'
```

```
'a' in my_str ⇒ True
```

```
'abb' in my_str ⇒ True
```

```
'x' in my_str ⇒ False
```

Basic String Operations

```
s = 'spam'
```

- length operator len()

```
len(s) ⇒ 4
```

- + is concatenate

```
new_str = 'spam' + '-' + 'spam-'
```

```
print(new_str) ⇒ spam-spam-
```

- * is repeat, the number is how many times

```
new_str * 3 ⇒
```

```
'spam-spam-spam-spam-spam-spam-'
```

Some Details

- Both `+` and `*` on strings makes a new string, does not modify the arguments
- Order of operation is important for concatenation, irrelevant for repetition
- The types required are specific. For concatenation you need two strings, for repetition a string and an integer

What Does $a + b$ Mean?

- What operation does the above represent? It depends on the types!
 - Two strings, concatenation
 - Two integers addition
- The operator $+$ is **overloaded**.
 - The operation $+$ performs depends on the types it is working on

String Comparisons, Single char

- Python 3 uses the Unicode mapping for characters.
 - Allows for representing non-English characters
- UTF-8, subset of Unicode, takes the English letters, numbers and punctuation marks and maps them to an integer.
- Single character comparisons are based on that number

Comparisons within Sequence

- It makes sense to compare within a sequence (lower case, upper case, digits).

– 'a' < 'b' → True

– 'A' < 'B' → True

– '1' < '9' → True

- Can be weird outside of the sequence

– 'a' < 'A' → False

– 'a' < '0' → False

Whole string comparison

- Compare the first element of each string
 - if they are equal, move on to the next character in each
 - if they are not equal, the relationship between those to characters are the relationship between the string
 - if one ends up being shorter (but equal), the shorter is smaller

Contents - String

Strings

String Index

String Slicing

String properties: Iterable, Immutable, Operations

String methods

Regular Expression

String Method

- a **method** is a variation on a function
 - like a function, it represents a program
 - like a function, it has input arguments and an output
- Unlike a function, it is applied in the context of a particular object.
- This is indicated by the dot notation invocation

Example 1

- `upper` is the name of a method. It generates a new string that has all upper case characters of the string it was called with.

```
my_str = 'Python Rules!'
```

```
my_str.upper() ⇒ 'PYTHON RULES!'
```

- The `upper()` method was called in the context of `my_str`, indicated by the dot between them.

More Dot Notation

- In generation, dot notation looks like:

– `object.method(...)`

- It means that the object in front of the dot is calling a method that is associated with that object's type.
- The method's that can be called are tied to the type of the object calling it. Each type has different methods.

Find

```
my_str = 'hello'
```

```
my_str.find('l')      # find index of 'l' in my_str
```

```
⇒ 2
```

Note how the method 'find' operates on the string object `my_str` and the two are associated by using the “dot” notation: `my_str.find('l')`.

Terminology: the thing(s) in parenthesis, i.e. the 'l' in this case, is called an **argument**.

Chaining Methods

Methods can be chained together.

- Perform first operation, yielding an object
- Use the yielded object for the next method

```
my_str = 'Python Rules!'
```

```
my_str.upper() ⇒ 'PYTHON RULES!'
```

```
my_str.upper().find('O')
```

```
⇒ 4
```

Optional Arguments

Some methods have optional arguments:

- If the user doesn't provide one of these, a default is assumed
- Find has a default second argument of 0, where the search begins

```
a_str = 'He had the bat'
```

```
a_str.find('t') ⇒ 7 # 1st 't', start at 0
```

```
a_str.find('t', 8) ⇒ 13 # 2nd 't'
```

Key String Methods

<code>count(sub[, start[, end]])</code>
<code>find(sub[, start[, end]])</code>
<code>index(<i>sub</i>[, <i>start</i>[, <i>end</i>]])</code>
<code>split([<i>sep</i> [,<i>maxsplit</i>]])</code>
<code>lower()</code>
<code>upper()</code>
<code>join(<i>seq</i>)</code>



List



Differences Between Lists and Strings

- lists can contain a mixture of any python object, strings can only hold characters
 - 1, “bill”, 1.2345, True
- lists are mutable, their values can be changed, while strings are immutable
- lists are designated with [], with elements separated by commas, strings use “ ” or ' '

Lists are mutable

```
>>> li = ['abc', 23, 4.34, 23]
>>> li[1] = 45
>>> li
['abc', 45, 4.34, 23]
```

- We can change lists *in place*.
- Name *li* still points to the same memory reference when we're done.

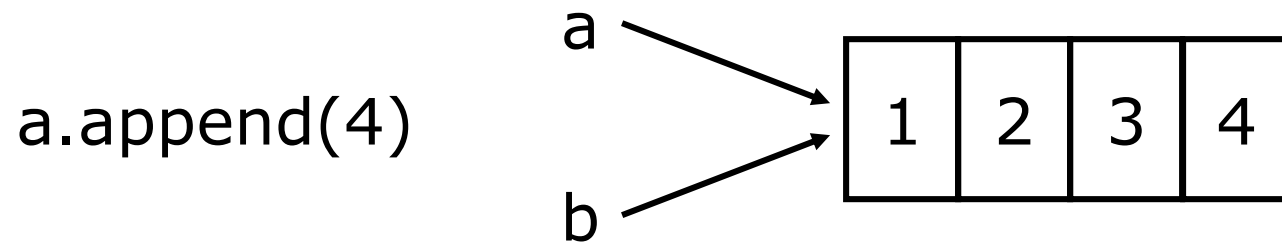
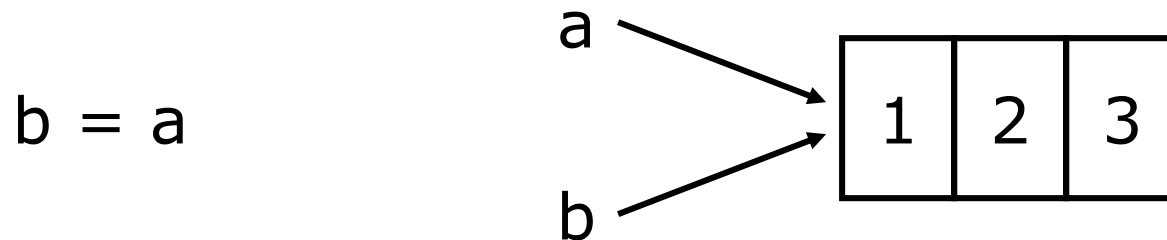
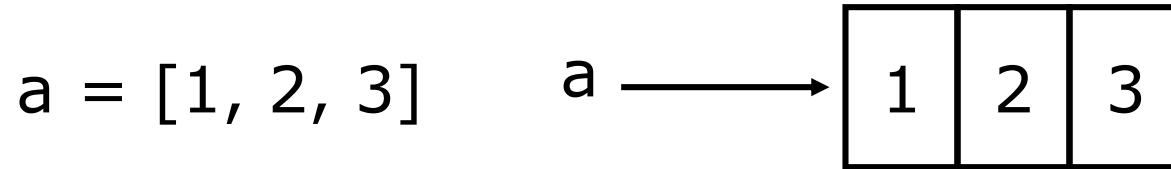
Some New Methods

- A list is mutable and can change:
 - `my_list[0]='a'     #index assignment`
 - `my_list.append(), my_list.extend()`
 - `my_list.pop()`
 - `my_list.insert(), my_list.remove()`
 - `my_list.sort()`
 - `my_list.reverse()`

Mutable vs Immutable

- Assignment manipulates references
 - `x = y` does not make a copy of the object `y` references
 - `x = y` makes `x` reference the object `y` references
- Very useful; but beware!, e.g.
 - `>>> a = [1, 2, 3] # a now references the list [1, 2, 3]`
 - `>>> b = a # b now references what a references`
 - `>>> a.append(4) # this changes the list a references`
 - `>>> print b # if we print what b references,`
`[1, 2, 3, 4] # SURPRISE! It has changed...`
- Why?

Why? Changing a Shared List



Copying the Whole Sequence

- `[ : ]` makes a *copy* of an entire sequence

```
>>> t[:]  
(23, 'abc', 4.56, (2,3), 'def')
```

- Note the difference between these two lines for mutable sequences

```
>>> l2 = l1 # Both refer to 1 ref,  
           # changing one affects both  
>>> l2 = l1[:] # Independent copies, two refs
```

Shallow Versus Deep

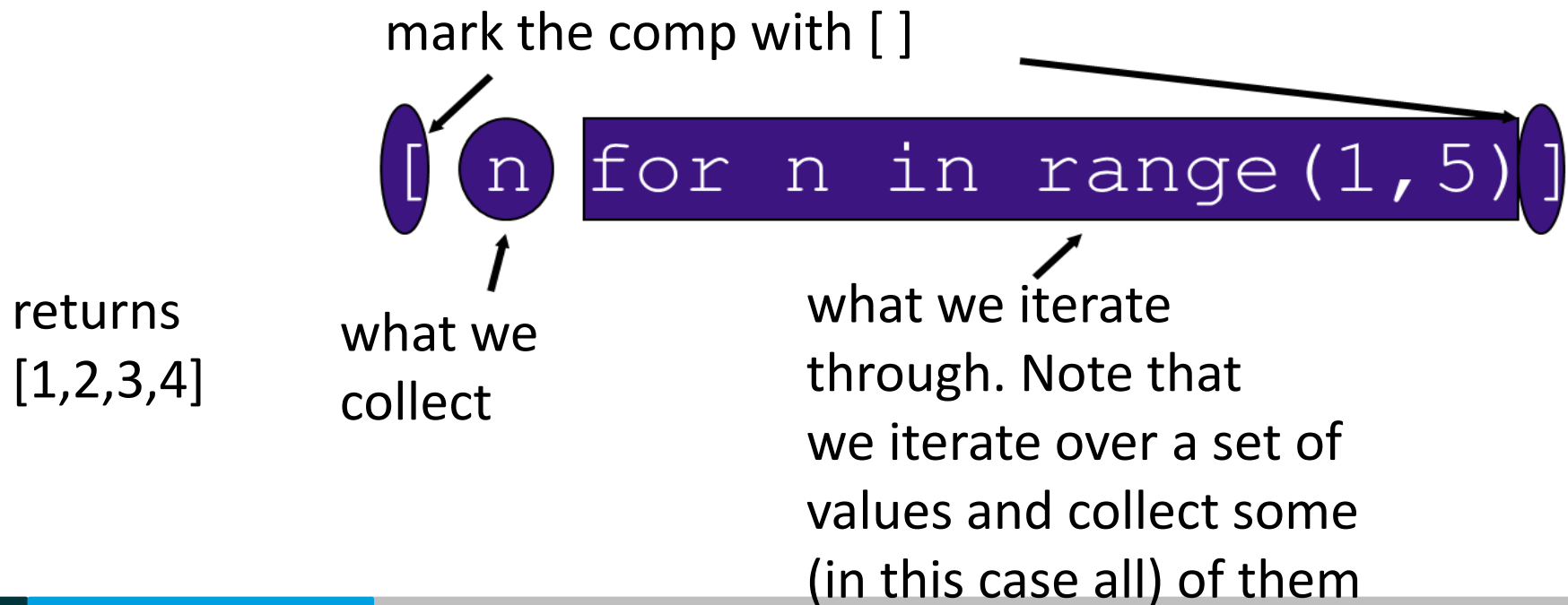
Regular copy, the `[:]` approach, only copies the top level reference/association
if you want a full copy, you can use `deepcopy`

```
>>> a_list = [1, 2, 3]
>>> b_list = [5, 6, 7]
>>> a_list.append(b_list)
>>> import copy
>>> c_list = copy.deepcopy(a_list)
>>> b_list[0]=1000
>>> a_list
[1, 2, 3, [1000, 6, 7]]
>>> c_list
[1, 2, 3, [5, 6, 7]]
>>>
```

Constructing lists

One way is a "list comprehension"

```
[n for n in range(1, 5)]
```



Modifying What We Collect

```
[ n**2 for n in range(1,6) ]
```

returns `[1, 4, 9, 16, 25]`. Note that we can only change the values we are iterating over, in this case `n`

Multiple Collects

```
[x+y for x in range(1,4) for y in range (1,4)]
```

It is as if we had done the following:

```
my_list = [ ]  
for x in range (1,4):  
    for y in range (1,4):  
        my_list.append(x+y)  
⇒ [2, 3, 4, 3, 4, 5, 4, 5, 6]
```

Modifying What Gets Collected

```
[c for c in "Hi There Mom" if c.isupper()]
```

- The `if` part of the comprehensive controls which of the iterated values is collected at the end. Only those values which make the if part true will be collected
⇒ ['H', 'T', 'M']



Tuple

Tuples

- Tuples are simply immutable lists
- They are printed with (,)

```
>>> 10,12          # Python creates a tuple
(10, 12)
>>> tup = 2,3      # assigning a tuple to a variable
>>> tup
(2, 3)
>>> (1)            # not a tuple, a grouping
1
>>> (1,)           # comma makes it a tuple
(1,)
>>> x,y = 'a',3.14159 # from on right, multiple assignments
>>> x
'a'
>>> y
3.14159
>>> x,y            # create a tuple
('a', 3.14159)
```

Tuples are immutable

```
>>> t = (23, 'abc', 4.56, (2,3), 'def')
>>> t[2] = 3.14
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#75>", line 1, in -toplevel-
    tu[2] = 3.14
```

```
TypeError: object doesn't support item assignment
```

- You can't change a tuple.
- You can make a fresh tuple and assign its reference to a previously used name.

```
>>> t = (23, 'abc', 3.14, (2,3), 'def')
```

- *The immutability of tuples means they're faster than lists.*

Summary: Tuples vs. Lists

- Lists slower but more powerful than tuples
 - Lists can be modified, and they have lots of handy operations and methods
 - Tuples are immutable and have fewer features
- To convert between tuples and lists use the `list()` and `tuple()` functions:

```
li = list(tu)
```

```
tu = tuple(li)
```

Reference

- Python Programming by Michael Dawson, ch4,5
- Punch & Enbody Chap 7